
Modul 1: Backend Core

Gün 01-30 - server, HTTP, API, database, cache, queue və performans

Bu paket backend developer üçün server əsasları, sistem dizaynı, arxitektura, dizayn pattern-lər, performans, etibarlılıq və production düşüncəsini ardıcıl öyrənmək üçün hazırlanıb. Hər gün bir mövzu oxu, həmin günün praktik tapşırığını yaz, sonra öz sistem dizayn qeydlərini yenilə.

Təvsiyə olunan gündəlik ritm: 60-90 dəqiqə nəzəriyyə, 60-120 dəqiqə praktika, 20 dəqiqə review.

Modulun istifadə qaydası

1. Hər günün əvvəlində məqsədi oxu və həmin mövzunu öz sözlərinlə izah etməyə çalış.
2. Nəzəriyyəni oxuduqdan sonra praktikanı mütləq kodla və ya dizayn sənədi ilə tamamla.
3. Mini-lab hissəsində ölçmə, test və ya failure scenario yarat. Production düşüncəsi burada formalaşır.
4. Müsahibə suallarına cavabı səsli izah et. Cavabın strukturunu sadə saxla: problem, seçim, trade-off, risk.

Gün 01: Backend developer üçün server mental modeli

Öyrənmə məqsədi

Günün sonunda 'Backend developer üçün server mental modeli' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Server tərəfini dərinləndirmək üçün əvvəlcə kodun işlədiyi mühiti görmək lazımdır: proses, thread, memory, file descriptor, socket, kernel queue və network stack. Backend kodu sadəcə controller və service deyil; o, OS resurslarını istifadə edən uzunömürlü prosesdir. Production problemi çıxanda CPU, memory, disk I/O, network latency və process lifecycle barədə düşünmək səni adi CRUD səviyyəsindən sistem mühəndisliyi səviyyəsinə keçirir.

Praktiki tətbiq

Kiçik HTTP servis yaz, eyni anda 100 request göndər, sonra process ID, open file descriptor sayı, CPU və memory istifadəsini ölç. Log-lara request ID əlavə et və bir request-in bütün mərhələlərini izləməyə çalış.

Mini-lab

Bir endpoint yarat: /health, /ready, /slow. /slow 2 saniyə gözləsin. Paralel request-lərlə serverin davranışını müşahidə et və thread/event-loop modelinin təsirini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Backend developer üçün server mental modeli real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 02: Linux prosesləri, file descriptor-lar və shell alətləri

Öyrənmə məqsədi

Günün sonunda 'Linux prosesləri, file descriptor-lar və shell alətləri' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Server tərəfini dərinləndirən anlamaq üçün əvvəlcə kodun işlədiyi mühiti görmək lazımdır: proses, thread, memory, file descriptor, socket, kernel queue və network stack. Backend kodu sadəcə controller və service deyil; o, OS resurslarını istifadə edən uzunömürlü prosesdir. Production problemi çıxanda CPU, memory, disk I/O, network latency və process lifecycle barədə düşünmək səni adi CRUD səviyyəsindən sistem mühəndisliyi səviyyəsinə keçirir.

Praktiki tətbiq

Kiçik HTTP servis yaz, eyni anda 100 request göndər, sonra process ID, open file descriptor sayı, CPU və memory istifadəsini ölç. Log-lara request ID əlavə et və bir request-in bütün mərhələlərini izləməyə çalış.

Mini-lab

Bir endpoint yarat: /health, /ready, /slow. /slow 2 saniyə gözləsin. Paralel request-lərlə serverin davranışını müşahidə et və thread/event-loop modelinin təsirini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Linux prosesləri, file descriptor-lar və shell alətləri real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 03: TCP/IP, port, socket və DNS

Öyrənmə məqsədi

Günün sonunda 'TCP/IP, port, socket və DNS' mövzusunun ham izah etməli, ham də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Şəbəkə backend sistemlərinin görünməyən müqaviləsidir. TCP etibarlı byte stream verir, amma latency, packet loss, retransmission və connection pooling qərarlarına həssasdır. DNS isə sadəcə ad çevirmə deyil; TTL, cache və failover davranışı ilə availability-ə təsir edir. Socket və port anlayışı olmadan load balancer, proxy və service discovery mövzuları yarımçıq qalır.

Praktiki tətbiq

nslookup, curl -v və netstat/Get-NetTCPConnection ilə real request-in hansı IP və porta getdiyini izlə. Keep-alive açıq və bağlı olanda latency fərqi ölç.

Mini-lab

Local serveri iki portda qaldır, sadə reverse proxy ilə traffic yönləndir və connection reuse davranışını logla.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. TCP/IP, port, socket və DNS real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 04: HTTP/1.1, HTTP/2, headers və status code-lar

Öyrənmə məqsədi

Günün sonunda 'HTTP/1.1, HTTP/2, headers və status code-lar' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

HTTP resource-lar üzərində semantik protokoldur: method-lar niyyəti, status code-lar nəticəni, header-lər metadata-nı ifadə edir. Backend dizaynında idempotency, cacheability, content negotiation, authentication header-ləri və error formatı API-nin istifadə rahatlığını müəyyən edir. HTTP-ni dərinləndirən bilmək debugging vaxtını ciddi azaldır.

Praktiki tətbiq

Eyni əməliyyatı POST və PUT ilə dizayn et, idempotent olub-olmadığını yaz. Error response üçün vahid format qur: code, message, details, requestId.

Mini-lab

curl ilə 200, 201, 204, 400, 401, 403, 404, 409, 422 və 429 qaytaran endpoint-lər yaz; hər birinin nə vaxt istifadə olunacağını README-də izah et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. HTTP/1.1, HTTP/2, headers və status code-lar real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 05: Web server, reverse proxy və load balancer fərqi

Öyrənmə məqsədi

Günün sonunda 'Web server, reverse proxy və load balancer fərqi' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Server tərəfini dərinləndirmək üçün əvvəlcə kodun işlədiyi mühiti görmək lazımdır: proses, thread, memory, file descriptor, socket, kernel queue və network stack. Backend kodu sadəcə controller və service deyil; o, OS resurslarını istifadə edən uzunömürlü prosesdir. Production problemi çıxanda CPU, memory, disk I/O, network latency və process lifecycle barədə düşünmək səni adi CRUD səviyyəsindən sistem mühəndisliyi səviyyəsinə keçirir.

Praktiki tətbiq

Kiçik HTTP servis yaz, eyni anda 100 request göndər, sonra process ID, open file descriptor sayı, CPU və memory istifadəsini ölç. Log-lara request ID əlavə et və bir request-in bütün mərhələlərini izləməyə çalış.

Mini-lab

Bir endpoint yarat: /health, /ready, /slow. /slow 2 saniyə gözləsin. Paralel request-lərlə serverin davranışını müşahidə et və thread/event-loop modelinin təsirini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Web server, reverse proxy və load balancer fərqi real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 06: Request lifecycle: browserdən database-ə qədər

Öyrənmə məqsədi

Günün sonunda 'Request lifecycle: browserdən database-ə qədər' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

HTTP resource-lar üzərində semantik protokoldur: method-lar niyyəti, status code-lar nəticəni, header-lər metadata-nı ifadə edir. Backend dizaynında idempotency, cacheability, content negotiation, authentication header-ləri və error formatı API-nin istifadə rahatlığını müəyyən edir. HTTP-ni dərinlən bilmək debugging vaxtını ciddi azaldır.

Praktiki tətbiq

Eyni əməliyyatı POST və PUT ilə dizayn et, idempotent olub-olmadığını yaz. Error response üçün vahid format qur: code, message, details, requestId.

Mini-lab

curl ilə 200, 201, 204, 400, 401, 403, 404, 409, 422 və 429 qaytaran endpoint-lər yaz; hər birinin nə vaxt istifadə olunacağını README-də izah et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Request lifecycle: browserdən database-ə qədər real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 07: Nginx/Envoy mental modeli və statik fayl serving

Öyrənmə məqsədi

Günün sonunda 'Nginx/Envoy mental modeli və statik fayl serving' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Server tərəfini dərinləndirmək üçün əvvəlcə kodun işlədiyi mühiti görmək lazımdır: proses, thread, memory, file descriptor, socket, kernel queue və network stack. Backend kodu sadəcə controller və service deyil; o, OS resurslarını istifadə edən uzunömürlü prosesdir. Production problemi çıxanda CPU, memory, disk I/O, network latency və process lifecycle barədə düşünmək səni adi CRUD səviyyəsindən sistem mühəndisliyi səviyyəsinə keçirir.

Praktiki tətbiq

Kiçik HTTP servis yaz, eyni anda 100 request göndər, sonra process ID, open file descriptor sayı, CPU və memory istifadəsini ölç. Log-lara request ID əlavə et və bir request-in bütün mərhələlərini izləməyə çalış.

Mini-lab

Bir endpoint yarat: /health, /ready, /slow. /slow 2 saniyə gözləsin. Paralel request-lərlə serverin davranışını müşahidə et və thread/event-loop modelinin təsirini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Nginx/Envoy mental modeli və statik fayl serving real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 08: REST API dizayn prinsipləri

Öyrənmə məqsədi

Günün sonunda 'REST API dizayn prinsipləri' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Yaxşı API daxili modeli olduğu kimi bayıra sızdırmır; stabil kontrakt təqdim edir. Resource adları, request validation, pagination, filtering, error taxonomy, versioning və backward compatibility uzunömürlü backend məhsullarında əsas mövzulardır. API dizaynı həm texniki, həm də istifadəçi təcrübəsidir: consumer kodunun sadə və səhvə davamlı olması hədəfidir.

Praktiki tətbiq

Bir domain seç: task, order və ya interview-question. CRUD endpoint-ləri OpenAPI ilə təsvir et, validation qaydalarını schema səviyyəsində yaz və breaking change nümunələri çıxar.

Mini-lab

Cursor-based pagination əlavə et. limit, nextCursor və deterministic ordering istifadə et. Offset pagination ilə fərqi test datası üzərində göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. REST API dizayn prinsipləri real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 09: API validation, error model və idempotency key

Öyrənmə məqsədi

Günün sonunda 'API validation, error model və idempotency key' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Yaxşı API daxili modeli olduğu kimi bayıra sızdırmır; stabil kontrakt təqdim edir. Resource adları, request validation, pagination, filtering, error taxonomy, versioning və backward compatibility uzunömürlü backend məhsullarında əsas mövzulardır. API dizaynı həm texniki, həm də istifadəçi təcrübəsidir: consumer kodunun sadə və səhvə davamlı olması hədəfidir.

Praktiki tətbiq

Bir domain seç: task, order və ya interview-question. CRUD endpoint-ləri OpenAPI ilə təsvir et, validation qaydalarını schema səviyyəsində yaz və breaking change nümunələri çıxar.

Mini-lab

Cursor-based pagination əlavə et. limit, nextCursor və deterministic ordering istifadə et. Offset pagination ilə fərqi test datası üzərində göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. API validation, error model və idempotency key real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 10: Pagination, filtering, sorting və search endpoint-ləri

Öyrənmə məqsədi

Günün sonunda 'Pagination, filtering, sorting və search endpoint-ləri' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Yaxşı API daxili modeli olduğu kimi bayıra sızdırmır; stabil kontrakt təqdim edir. Resource adları, request validation, pagination, filtering, error taxonomy, versioning və backward compatibility uzunömürlü backend məhsullarında əsas mövzulardır. API dizaynı həm texniki, həm də istifadəçi təcrübəsidir: consumer kodunun sadə və səhvə davamlı olması hədəfdir.

Praktiki tətbiq

Bir domain seç: task, order və ya interview-question. CRUD endpoint-ləri OpenAPI ilə təsvir et, validation qaydalarını schema səviyyəsində yaz və breaking change nümunələri çıxar.

Mini-lab

Cursor-based pagination əlavə et. limit, nextCursor və deterministic ordering istifadə et. Offset pagination ilə fərqi test datası üzərində göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Pagination, filtering, sorting və search endpoint-ləri real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 11: API versioning və backward compatibility

Öyrənmə məqsədi

Günün sonunda 'API versioning və backward compatibility' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Yaxşı API daxili modeli olduğu kimi bayıra sızdırmır; stabil kontrakt təqdim edir. Resource adları, request validation, pagination, filtering, error taxonomy, versioning və backward compatibility uzunömürlü backend məhsullarında əsas mövzulardır. API dizaynı həm texniki, həm də istifadəçi təcrübəsidir: consumer kodunun sadə və səhvə davamlı olması hədəfidir.

Praktiki tətbiq

Bir domain seç: task, order və ya interview-question. CRUD endpoint-ləri OpenAPI ilə təsvir et, validation qaydalarını schema səviyyəsində yaz və breaking change nümunələri çıxar.

Mini-lab

Cursor-based pagination əlavə et. limit, nextCursor və deterministic ordering istifadə et. Offset pagination ilə fərqi test datası üzərində göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. API versioning və backward compatibility real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 12: Authentication: session, JWT, OAuth2 əsasları

Öyrənmə məqsədi

Günün sonunda 'Authentication: session, JWT, OAuth2 əsasları' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Security backend-də sonradan əlavə olunan qat deyil; hər request üçün identity, permission, input trust və data exposure sualları verilməlidir. Authentication kim olduğunu, authorization nə edə biləcəyini müəyyən edir. Secrets, audit log, rate limit və threat modeling production sistemlərində minimum gözləntidir.

Praktiki tətbiq

Endpoint-ləri public, authenticated və owner-only qruplarına böl. Hər endpoint üçün kim, hansı resource-a, hansı şərtlə access alır cədvəlini yaz.

Mini-lab

JWT və session əsaslı login-in trade-off-larını müqayisə et. Token expiry, refresh, logout və compromised token senarilərini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Authentication: session, JWT, OAuth2 əsasları real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 13: Authorization: RBAC, ABAC və resource ownership

Öyrənmə məqsədi

Günün sonunda 'Authorization: RBAC, ABAC və resource ownership' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Security backend-də sonradan əlavə olunan qat deyil; hər request üçün identity, permission, input trust və data exposure sualları verilməlidir. Authentication kim olduğunu, authorization nə edə biləcəyini müəyyən edir. Secrets, audit log, rate limit və threat modeling production sistemlərində minimum gözləntidir.

Praktiki tətbiq

Endpoint-ləri public, authenticated və owner-only qruplarına böl. Hər endpoint üçün kim, hansı resource-a, hansı şərtlə access alır cədvəlini yaz.

Mini-lab

JWT və session əsaslı login-in trade-off-larını müqayisə et. Token expiry, refresh, logout və compromised token senarilərini qeyd et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Authorization: RBAC, ABAC və resource ownership real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 14: OpenAPI, contract testing və SDK düşüncəsi

Öyrənmə məqsədi

Günün sonunda 'OpenAPI, contract testing və SDK düşüncəsi' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Yaxşı API daxili modeli olduğu kimi bayıra sızdırmır; stabil kontrakt təqdim edir. Resource adları, request validation, pagination, filtering, error taxonomy, versioning və backward compatibility uzunömürlü backend məhsullarında əsas mövzulardır. API dizaynı həm texniki, həm də istifadəçi təcrübəsidir: consumer kodunun sadə və səhvə davamlı olması hədəfidir.

Praktiki tətbiq

Bir domain seç: task, order və ya interview-question. CRUD endpoint-ləri OpenAPI ilə təsvir et, validation qaydalarını schema səviyyəsində yaz və breaking change nümunələri çıxar.

Mini-lab

Cursor-based pagination əlavə et. limit, nextCursor və deterministic ordering istifadə et. Offset pagination ilə fərqi test datası üzərində göstər.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. OpenAPI, contract testing və SDK düşüncəsi real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 15: Relational data modeling və normalization

Öyrənmə məqsədi

Günün sonunda 'Relational data modeling və normalization' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Relational data modeling və normalization real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 16: SQL query plan, indeks və selectivity

Öyrənmə məqsədi

Günün sonunda 'SQL query plan, indeks və selectivity' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. SQL query plan, indeks və selectivity real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 17: Transaction, ACID və isolation level-lər

Öyrənmə məqsədi

Günün sonunda 'Transaction, ACID və isolation level-lər' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Transaction, ACID və isolation level-lər real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 18: Locking, deadlock və optimistic concurrency

Öyrənmə məqsədi

Günün sonunda 'Locking, deadlock və optimistic concurrency' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırarsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Locking, deadlock və optimistic concurrency real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 19: Migration strategiyası və zero-downtime schema change

Öyrənmə məqsədi

Günün sonunda 'Migration strategiyası və zero-downtime schema change' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Migration strategiyası və zero-downtime schema change real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 20: NoSQL modelləri: document, key-value, wide-column

Öyrənmə məqsədi

Günün sonunda 'NoSQL modelləri: document, key-value, wide-column' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. NoSQL modelləri: document, key-value, wide-column real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 21: Database backup, restore və PITR düşüncəsi

Öyrənmə məqsədi

Günün sonunda 'Database backup, restore və PITR düşüncəsi' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Database dizaynı backend-in uzunmüddətli elastikliyi müəyyən edir. Schema yalnız data saxlamaq üçün deyil; invariant-ları qorumaq, query-ləri effektiv etmək və transaction sərhədlərini aydınlaşdırmaq üçündür. İndeks seçimi read performansını artırsa da write cost və storage yaradır. Isolation səviyyələri isə correctness və throughput arasında seçimdir.

Praktiki tətbiq

Order, OrderItem, Payment və User cədvəlləri qur. Əsas query-lər üçün indeks təklif et, sonra EXPLAIN planla həqiqətən istifadə olunub-olunmadığını yoxla.

Mini-lab

İki paralel transaction ilə lost update və deadlock nümunəsi yarat. Optimistic locking üçün version column əlavə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Database backup, restore və PITR düşüncəsi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 22: Redis və cache strategiyaları

Öyrənmə məqsədi

Günün sonunda 'Redis və cache strategiyaları' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Cache performans alətidir, amma correctness riski gətirir. Cache-aside, read-through, write-through və write-behind strategiyaları fərqli consistency davranışı yaradır. TTL, invalidation, stampede protection və key design bilmədən Redis əlavə etmək sistemi sürətləndirmək əvəzinə anlaşılmaz bug-lar yarada bilər.

Praktiki tətbiq

Tez-tez oxunan endpoint üçün cache-aside tətbiq et. Cache hit/miss metrikası yaz və TTL seçiminin istifadəçi təcrübəsinə təsirini ölç.

Mini-lab

Eyni anda 100 request cache miss yaratdıqda database-ə neçə query getdiyini ölç. Lock və ya single-flight mexanizmi ilə stampede-i azalt.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Redis və cache strategiyaları real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 23: Cache invalidation, stampede və TTL seçimi

Öyrənmə məqsədi

Günün sonunda 'Cache invalidation, stampede və TTL seçimi' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Cache performans alətidir, amma correctness riski gətirir. Cache-aside, read-through, write-through və write-behind strategiyaları fərqli consistency davranışı yaradır. TTL, invalidation, stampede protection və key design bilmədən Redis əlavə etmək sistemi sürətləndirmək əvəzinə anlaşılmaz bug-lar yarada bilər.

Praktiki tətbiq

Tez-tez oxunan endpoint üçün cache-aside tətbiq et. Cache hit/miss metrikası yaz və TTL seçiminin istifadəçi təcrübəsinə təsirini ölç.

Mini-lab

Eyni anda 100 request cache miss yaratdıqda database-ə neçə query getdiyini ölç. Lock və ya single-flight mexanizmi ilə stampede-i azalt.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Cache invalidation, stampede və TTL seçimi real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 24: Rate limiting və quota dizaynı

Öyrənmə məqsədi

Günün sonunda 'Rate limiting və quota dizaynı' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Performans optimizasiyası təxminlə deyil, ölçmə ilə başlayır. Latency percentiles, throughput, saturation və error rate birlikdə oxunmalıdır. Bottleneck CPU, DB, lock contention, network və ya external dependency ola bilər. Yaxşı backend developer ölçür, hipotez qurur, kiçik dəyişiklik edir və yenidən ölçür.

Praktiki tətbiq

Load test planı yaz: normal load, peak load və stress load. p50, p95, p99 latency və error rate ölç.

Mini-lab

N+1 query problemi yarat, sonra eager loading və ya batch query ilə düzəlt. Hər iki halın latency və query sayını müqayisə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Rate limiting və quota dizaynı real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 25: Background jobs, queue və worker pool

Öyrənmə məqsədi

Günün sonunda 'Background jobs, queue və worker pool' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Queue və broker-lər sistemə elastiklik verir: request-i tez qəbul edib işi background-a ötürmək, spike-ları hamarlamaq və servis sərhədlərini zəiflətmək mümkündür. Amma delivery semantics, retry, dead-letter queue, ordering və idempotent consumer dizayn edilməlidir. Async sistemlərdə 'mesaj bir dəfə gələcək' fərziyyəsi təhlükəlidir.

Praktiki tətbiq

Email göndərmə işini synchronous endpoint-dən çıxarıb queue-ya at. Worker retry və DLQ davranışını logla.

Mini-lab

Eyni mesajı iki dəfə emal edən consumer yaz və idempotency key ilə duplicate təsirini aradan qaldır.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Background jobs, queue və worker pool real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 26: Message broker: RabbitMQ, Kafka və semantics

Öyrənmə məqsədi

Günün sonunda 'Message broker: RabbitMQ, Kafka və semantics' mövzusunun həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Queue və broker-lər sistemə elastiklik verir: request-i tez qəbul edib işi background-a ötürmək, spike-ları hamarlamaq və servis sərhədlərini zəiflətmək mümkündür. Amma delivery semantics, retry, dead-letter queue, ordering və idempotent consumer dizayn edilməlidir. Async sistemlərdə 'mesaj bir dəfə gələcək' fərziyyəsi təhlükəlidir.

Praktiki tətbiq

Email göndərmə işini synchronous endpoint-dən çıxarıb queue-ya at. Worker retry və DLQ davranışını logla.

Mini-lab

Eyni mesajı iki dəfə emal edən consumer yaz və idempotency key ilə duplicate təsirini aradan qaldır.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Message broker: RabbitMQ, Kafka və semantics real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 27: Async processing, outbox pattern və eventual consistency

Öyrənmə məqsədi

Günün sonunda 'Async processing, outbox pattern və eventual consistency' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Queue və broker-lər sistemə elastiklik verir: request-i tez qəbul edib işi background-a ötürmək, spike-ları hamarlamaq və servis sərhədlərini zəiflətmək mümkündür. Amma delivery semantics, retry, dead-letter queue, ordering və idempotent consumer dizayn edilməlidir. Async sistemlərdə 'mesaj bir dəfə gələcək' fərziyyəsi təhlükəlidir.

Praktiki tətbiq

Email göndərmə işini synchronous endpoint-dən çıxarıb queue-ya at. Worker retry və DLQ davranışını logla.

Mini-lab

Eyni mesajı iki dəfə emal edən consumer yaz və idempotency key ilə duplicate təsirini aradan qaldır.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Async processing, outbox pattern və eventual consistency real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 28: Performance profiling və bottleneck tapmaq

Öyrənmə məqsədi

Günün sonunda 'Performance profiling və bottleneck tapmaq' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Performans optimizasiyası təxminlə deyil, ölçmə ilə başlayır. Latency percentiles, throughput, saturation və error rate birlikdə oxunmalıdır. Bottleneck CPU, DB, lock contention, network və ya external dependency ola bilər. Yaxşı backend developer ölçür, hipotez qurur, kiçik dəyişiklik edir və yenidən ölçür.

Praktiki tətbiq

Load test planı yaz: normal load, peak load və stress load. p50, p95, p99 latency və error rate ölç.

Mini-lab

N+1 query problemi yarat, sonra eager loading və ya batch query ilə düzəlt. Hər iki halın latency və query sayını müqayisə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Performance profiling və bottleneck tapmaq real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 29: Load testing, capacity planning və SLO

Öyrənmə məqsədi

Günün sonunda 'Load testing, capacity planning və SLO' mövzusunı həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Performans optimizasiyası təxminlə deyil, ölçmə ilə başlayır. Latency percentiles, throughput, saturation və error rate birlikdə oxunmalıdır. Bottleneck CPU, DB, lock contention, network və ya external dependency ola bilər. Yaxşı backend developer ölçür, hipotez qurur, kiçik dəyişiklik edir və yenidən ölçür.

Praktiki tətbiq

Load test planı yaz: normal load, peak load və stress load. p50, p95, p99 latency və error rate ölç.

Mini-lab

N+1 query problemi yarat, sonra eager loading və ya batch query ilə düzəlt. Hər iki halın latency və query sayını müqayisə et.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsahibə və özünü yoxlama sualları

1. Load testing, capacity planning və SLO real production sisteminə hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.

Gün 30: Module 1 capstone: production-ready CRUD service

Öyrənmə məqsədi

Günün sonunda 'Module 1 capstone: production-ready CRUD service' mövzunu həm izah etməli, həm də kiçik backend nümunəsində tətbiq etməlisən.

Dərin nəzəri izah

Capstone günlərinin məqsədi ayrı-ayrı bilikləri bir sistemdə birləşdirməkdir. Burada cavabın gözəlliyi diaqramda deyil; requirements, edge case, data model, API, scaling, consistency və failure handling-in bir-birinə uyğun olmasındadır.

Praktiki tətbiq

Sistemi əvvəl monolith kimi dizayn et, sonra hansı sərhədlərin service-ə çevrilə biləcəyini əsaslandır. Hər qərara trade-off yaz.

Mini-lab

30 dəqiqəlik mock interview et: 5 dəqiqə requirements, 5 dəqiqə estimation, 10 dəqiqə architecture, 5 dəqiqə bottleneck, 5 dəqiqə reliability review.

Dərinləşdirmə qeydləri

- Əvvəlcə konsepti öz sözlərinlə 5-7 cümləlik izah et. Əgər izah qarışıqdırsa, kod yazmağa keçmə.
- Praktikanı kiçik saxla, amma ölçülə bilən et: log, metrika, test, query plan və ya request trace əlavə et.
- Hər günün sonunda 'bu qərarın trade-off-u nədir?' sualına ən azı iki cavab yaz.

Müsaibə və özünü yoxlama sualları

1. Module 1 capstone: production-ready CRUD service real production sistemində hansı problemi həll edir?
2. Bu mövzuda ən çox edilən səhv nədir və onu necə aşkar edərsən?
3. Bu qərar latency, availability, consistency və maintainability-yə necə təsir edir?

Günün çıxışı

Bu günün sonunda ən azı bir işləyən kod nümunəsi, bir ölçmə nəticəsi və 5-10 cümləlik şəxsi izah yazmalısən. Əgər mövzunu başqa backend developer-ə izah edə bilmirsənsə, növbəti günə keçməzdən əvvəl labı təkrar et.